

17/8/26

classmate

Date

Page

Encapsulation:

Bundling data & operations together while controlling the access to internal state.

Private

```
class classname
    DT Variable = Value
    int a = "
    int b = "
    obj. classname
    constructor - this. a = ?
```

Public { class extends / implement  
getter  
setter

getter doesn't change anything.  
Use setter to change.

abstraction x access the parent class it access only in encapsulation.

write a program to create a classname student include name, age.

Encapsulation

class student {

string name; "David"

int age; "21"

public void setName ("David") {

this.name = name;

}

public void setage ("21") {

this.age = age;

}

public string getName () {

return name

}

classmate  
Date \_\_\_\_\_  
Page \_\_\_\_\_

```

public int getAge() {
    return age;
}

public void show() {
    System.out.println("name: " + name);
    System.out.println("age: " + age);
}

public static void main(String[] args) {
    Student s1 = new Student("David");
    s1.setName("David");
    s1.setAge("18");
    s1.show();
}

```

### Abstraction:-

process of eliminating not essential details. D. Types. so that only the necessary structural features of an entity remain visible to the caller.

It establishes a clear boundary b/w the specification.

### Interface:-

An interface in java pure structural contract that defines structural behaviours and independent implementing. Class must commit to, interfaces cannot be instantiated directly.

A class can implement multiple interface by passing a single inheritance relationship of classes. All fields declared inside & p interface are public static final.

```

classA (int x, int y) {
    this.x = x;
    this.y = y;
}

```

changed.

All method signatures declares without

Instance variable defined inside class but outside any method constructor and code blocks.

An abstract class is a class marked with a `abstract` keyword that can't be directly instantiated. It acts as a partial blueprint intended to be extended by concrete subclasses.

Local & Global  
Instance & Abstract  
Local & Instance

Rules:

1. → It has no method body.
- Abstract method must not contain curly braces.
2. → Enclosing class constraints, an abstract method can only access inside an abstract class or an interface.
3. Mandatory overriding  
1st concrete subclass extending an abstract parent class must override & provide concrete implementation for all inherited abstract methods.  
Abstract can't be declared as `public`, `private`, `protected`.

→ `abstract`

```

public abstract class A {
    abstract x, y;
}

```

→ Instance variable

```

public abstract class A {
    public abstract void print x, y;
}

```

Abstract method:

public abstract void main() {  
[public can be declared when an instance  
object is not created.]

If any object is created, then the  
abstract method uses abstract void main()  
without public.

Sorter, Strategy, Pattern - Merge sort.

- Create an abstract class shape that  
represent geometry shapes.
- class name shld be in Shape
- abstract method called double Area()
- " " double perimeter()
- concrete method void display()
- Create another class circle, radius.

```
abstract class shape {  
    abstract double Area();  
    abstract double perimeter();  
}
```

```
class circle extends shape {  
    double r = 5;  
}
```

```
    double area() {  
        return 3.14 * r * r;
```

```
    double perimeter() {  
        return 2 * 3.14 * r;
```

```
    void display() {  
        System.out.println ("Area = " + Area());
```

classmate  
Date \_\_\_\_\_  
Page \_\_\_\_\_

```

    } System.out.println("Perimeter = " + perimeter);
    }

    public class main
    {
        public static main (String[] args)
        {
            circle = new circle();
            c.display();
        }
    }

```

### Inheritance:

It is a mechanism where in child class derives state and behaviour from a parent class using the extends keyword. It establishes and is-a, has-a relationship. means it contains

Single → class B extends class A.

Multiple → class C extends class B, class B extends class A.

hierarchical → class C extends class B & class A.

→ Upcasting & downcasting.

Person 1 & Person 2.

Rollno.

Name

Age.

classmate  
Date \_\_\_\_\_  
Page \_\_\_\_\_

```
public class Student {  
    int rollno;
```

```
    {  
        class personOne {  
            String name;  
            int age;  
        }  
    }
```

```
    *class personTwo extends personOne {  
        String name;  
        int age; }*
```

```
    {  
        personOne (String name, int age) {  
            this.name = name;  
            this.age = age;  
        }  
    }
```

```
    {  
        class personTwo extends personOne {  
            // constructor  
            personTwo (String name, int age) {  
                super (name, age);  
            }  
        }  
    }
```

```
public static void main (String[] args) {  
    Student s1 = new Student ("Seeriyaa", 12);  
    s1.rollno =
```

Overloading:- Multiple methods with the same name but different parameters in the same class

```
class student {  
    student() {  
    }  
    student (String name) {  
        this.name = name;  
    }  
}
```

Keyword represent current object.

Overriding:-

child class provides its own implementation of a method that it is already define in parent class.

Polymorphism:-

It is the capability of single entity [method calling or interface references] to exhibit different behaviours depending on context or the underlying object type.

Types:-

- 1 Static - Compile time, Overloading
  - 2 Dynamic - Run time, Overriding
- Zero run time } Binding time.

(Parameter, type, number) - usage

Dynamic methods -> object() - stored in heap memory  
usage (pointers)

Compile time polymorphism - Method overloading  
Method overloading occurs when single class defines multiple methods sharing the same

but different parameters.  
Overloading can't be achieved solely by  
changing the return type of the method.

Add two numbers / Sum of numbers

a =

b =

S.O.P (" ", a+b);

But using parameter constructor,

```
public int add (inta, intb)
{
    return (a+b);
}
```

Method overriding :-

It occurs when child class provides an implementation of the method that already declared in parent class. When calling an overridden method through a parent reference java evaluates the actual time on the heap at run time rather than the reference time.

Write a program using override method with properties play & sounds.

```
class Animal {
```

```
    void play() {
```

```
        System.out.println("Animal is playing.");
```

```
    }
```

void sound() {

system.out.println(x: "Animal makes sound");

Control flow: (Executes statement)

statements execute in which it has sequential execution top to bottom.

int a = 10

int b = 10

int sum = a + b;

P.out(sum)

Control flow lets a program make decision & executes different blocks of code based on conditions. (true or false)

\* If, If else, else if, switch

If statement:

It executes the block when the condition is true.

Syntax: if (Condition) {  
          - - -  
          }  
          int age = 20;  
          if (age = 20) {  
              s.o.println("Eligible")  
          }

Ex: if (age = 20)  
      if (age > 20)  
          s.o.println("Eligible")  
      } else {

s.o.println("not eligible")

If-else:

Provides an alternative path where condition is false.

```
if (Condition)
{
    - - -
}
else {
    - - -
}
```

classmate  
Date \_\_\_\_\_  
Page \_\_\_\_\_

If-else-if ladder:-  
checks multiple <sup>conditions in</sup> sequence, only the 1st  
2 blocks runs.

```
if (condition) {  
    // ...  
}  
else if (condition) {  
    // ...  
}  
else if (conditions) {  
    // ...  
}
```

Switch :-

Switch expression is evaluated once,  
the result is compared with each case value.  
It is used instead of multiple if statements.  
If there is a match, the matching block  
of code runs.

\* The break statements stops the switch  
after the matching case has run.  
The default statement runs if there is no  
match.

```
Switch (day) {  
    case 1: {  
        // ...  
    }  
    // ...  
}
```

Practise program:  
Prime number:

```
int n = 7, i;  
for (i = 2; i < n; i++)  
    if (n % i == 0)  
        break;  
S.O. println (i == n ? "Prime" : "Not Prime");  
}
```

Stack: The no. of iteration known in advance.

Loop: It executes block of code repeatedly based on condition. point odd & even, given no. is 1 to 20

```
public class evenodd {  
    public static void main (String[] args) {  
        S.O. Pn ("Even numbers:");  
        for (int i = 1; i <= 20; i++) {  
            if (i % 2 == 0)  
                S.O. P (i + " ");  
        }  
        S.O. Pn ("Odd numbers");  
        for (int i = 1; i <= 20; i++) {  
            if (i % 2 != 0)  
                S.O. P (i + " ");  
        }  
    }  
}
```

Methods:-

A method is a reusable block of code that encapsulation a single response.

Recursion:-

Recursion occurs when a method calls itself to solve a problem by working on a smaller instance of that same problem.

Java run time uses a call stack to track method.

Every call pushes new frame on to a stack.

When a method returns its frame is popped and control returns to the caller.

call stack LIFO.

public

→ Generics :-

Generic lets classes, interfaces & methods operate on a type parameters specified by the caller giving compile time <sup>type safety</sup> & eliminating manual casting.

19/8/2026.

→ Collection frameworks:-

that provide set of classes & interfaces. ^ readymade data structure to store and manipulate group of object.

Collection is an object that can store multiple elements in a single unit.

Collection provides interfaces defines what a data structure can do

ArrayList, LinkedList  
Collection Collections

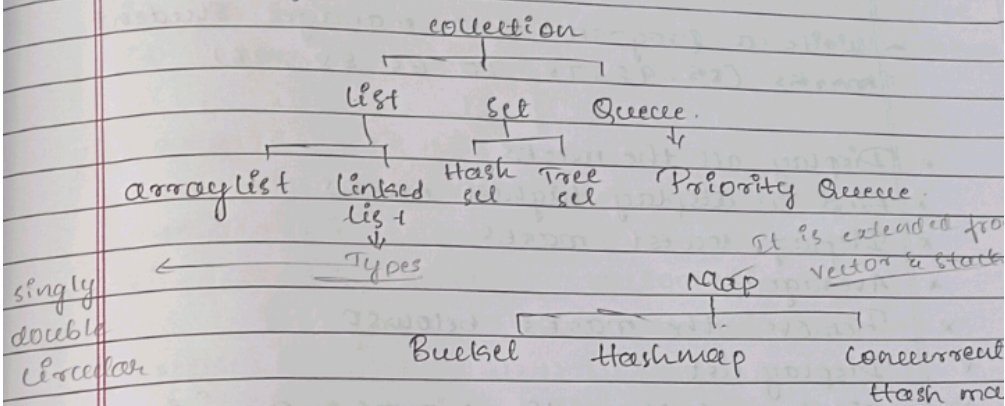
Syntax:

List < Patients  
= new ArrayList < > ();

Actual classes implement those interfaces is called implementations. Mechanism for traversal collection is called Iterators.

DSA

Hierarchy



Wrapper class:

It is a class whose object wraps or encapsulates primitive data types.

Byte < Bytes  
int < Integer  
char < Character  
float < float

List Interface:

4-usage-maintain Supports  
List represents an ordered collection. It allows insertion order. It allows index based access.

Element can be accessed using index.

ArrayList is one of the most frequently used java collections in DSA

an ArrayList is Backed internally with a dynamic array. unlike a normal array it can grow dynamically.

→ ArrayList <Integer> numbers  
= new ArrayList <>()

→ Write a program that manages student marks (85, 92, 78, 90, 65, 88, 76)

- \* Display all the marks
- \* Find & display highest marks
- \* Display lowest marks
- \* Average
- \* Remove the marks below 80
- \* Display list
- \* Check whether the marks 90 is removed or not → Boolean

Linked list:

It implements 3 interfaces at which is list, queue, deque, which is why, it shows up in so many different rows. Internally it is doubly linked list.

Each node stores 3 things & refers to the previous node data & refers to the next node.

## ArrayList

- \* Dynamic array
- \* Fast
- \* Less
- \* Insertion O(n)

## LinkedList

- \* Double linked list
- \* Slow
- \* memory more overhead
- \* O(n)

## Set Interface :-

A set represents collection of unique elements

Set interface extends the collection interface  
It is available in the java.util package  
It doesn't allow duplicate elements

## HashSet :-

Use a HashSet when u need fast membership checking and uniqueness  
It doesn't order automatically

```
Set <Integer> set = new HashSet <> ();
```

## TreeSet :-

Stores unique elements in sorted order

```
Set <Integer> name = new TreeSet <> ();
```

## Queue Interface (FIFO) :-

Offer() → add

Poll() → Remove

Peek() → Return first element

Create a simple student result management system where student roll no is 3.  
studentname, marks or value.

Perform adding, searching, displaying, removing, updating

classmate  
Date \_\_\_\_\_  
Page \_\_\_\_\_

Map:  
Map stores data as key value pairs.

Immutable vs Modifiable collections.

Tree map:  
Keeps it ~~tree~~ <sup>keys</sup> in sorted order  
at all the types like  $\{ \}$  set  $\{ \}$  map  
implements navigable interface with the  
same family of neighbour finding methods  
& operating on keys.

`map.floorKey(25)`  $\rightarrow$  highest element in a map  
`map.ceilingKey(25)`  $\rightarrow$  lowest  
`map.firstKey(25)`  $\rightarrow$  first  
`map.headMap(10)`  $\rightarrow$  all elements until 10  
`map.tailMap(5)`  $\rightarrow$

Comparable:  
It defines a classes single nature  
ordering implemented inside the class  
itself.

Comparator:  
It defines an external swapping  
ordering useful when u need several  
different ordering for the same class.

Exception:  
An exception is an abnormal condition  
that occurs during the execution of  
the program & Disrupts the normal  
expected flow of instructions.

Throwable:

It is the superclass of all objects that can be thrown by java exception. every error & every exception ultimately extends throwable. It provides methods such as `getMessage()` & `toString()`.

Thread:

20/08/26

Multithreading is the process of executing multiple threads concurrently within a single program.

Thread is the smallest unit of execution within a process.

Process Vs Thread

Process is a running program with its own independent isolated block of memory that other process cannot directly see into.

Eg: Chrome

Thread is an execution path

If everything happens sequential the application may appear slow. the user is stuck waiting for the download to finish completely before processing even starts. & waiting for processing to finish before the UI updates at all. even though these are conceptually fairly independent activities.

### Thread lifecycle.

#### Race condition

It occurs when multiple threads access shared to mutable data concurrently & the final result depends on the order of their execution.

#### Deadlock

A Deadlock occurs when two or more threads wait indefinitely for resources held by each other.

If race condition is a case of threads interfering with sharing data, a deadlock is a case of threads interfering with each other's progress. Neither thread does anything wrong individually but together they lock the program permanently.

Synchronisation is the mechanism to control the access of multiple thread to share the resource. It is essential in multithreaded environment to prevent thread interference and consistency problems.

Synchronisation ensures that only one thread access the shared resources at a time.

Writing a correct program answers only one question. A second equally important question is how efficiently the program teaches that as the input grows.

Complexity analysis is the discipline of answering that second question before you even run the code.

Ex: Binary, linear search

Two measures of complexity

- \* Time complexity
- \* Space complexity

Time Complexity:-

It is not literally how many seconds does it takes. It describes the growth rate of computational work related to input size.

Asymptotic [notations] Analysis:-

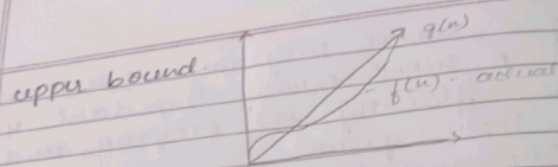
- \* Worst - when it takes huge time.
- \* Best - lowest time.
- \* Average - Average time.

Calculated based on runtime complexity

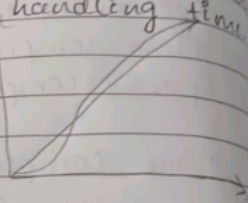
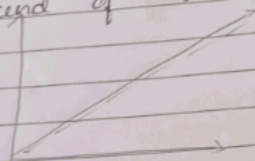
Big O:-

This measures the performance of algorithm, by simply providing the growth of the func.

It provides upper bound func: ensures that func: never grows faster.



$\Omega$  Omega: It basically describes test case scenario which is opposite to Big O notation. It is the formal way of representing lower bound of the algorithm handling time.



$\Theta$  Theta: It mainly describes average case.

Space complexity: Only a fixed no. of extra variables are used regardless of  $n$ .

Additional space (Big)  $O(n)$

→ Given numbers =  $[2, 7, 11, 15]$  target = 9.  
find the indices of 2 no.'s added to the target.

→ Write a program to calculate using modular exponential values of  $a$  &  $b$  very large, so the program should avoid directly calculating  $a^b$ .

```
for (i = -1; i <= 100; ++i) } Time Complexity &  
print ("Hello, world"); } Space Complexity.
```

```
for (int i = 0; i < nums.length; i++) {  
    for (int j = i + 1; j < nums.length; j++) {  
        if (nums[i] + nums[j] == target) {  
            return new int[] {  
                i, j  
            };  
        }  
    }  
}  
return new int[] {}
```